

ASSESSMENT TOOL

CO2 AT-1

Name : Arthi. A

Reg no : 192525152

Course code : CSA0307

Course name : Data Structures

Date : 26.7.2026

- 1) Insert an element at 3rd position in doubly linked list, having 6 elements in list and delete the element at 2nd position from list.

Answer :

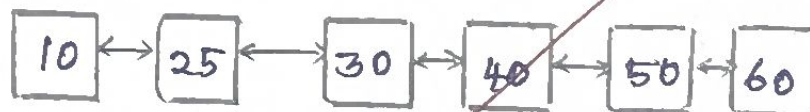
original list - 6 nodes



Insert 25 at position 3



Delete node at position 2



Insert 25 at position 3 :

1. new $\rightarrow$ prev = node 20
2. new $\rightarrow$ next = node 30
3. node 20 $\rightarrow$ next = new
4. node 30 $\rightarrow$ prev = new

Arthi
27/7/26

Delete node at position 2 :

1. node 10 $\rightarrow$ next = node 25
2. node 25 $\rightarrow$ prev = node 10
3. free (node 20)

C Implementation :

```
#include <stdio.h>
#include <stdlib.h>
```

```
struct node {
```

```
    int data;
```

```
    struct node *prev, *next;
```

```
};
```

```
struct node * createNode(int val) {
```

```
    struct node * n = (struct Node *) malloc(sizeof(struct node));
```

```
    n->data = val;
```

```
    n->prev = n->next = NULL;
```

```
    return n;
```

```
}
```

```
struct Node * insertatposition(struct node * head, int val,
```

```
    struct node * newNode = createNode(val);    int pos) {
```

```
    if (pos == 1) {
```

```
        newNode->next = head;
```

```
        if (head != NULL) head->prev = newNode;
```

```
        return newNode;
```

```
    }
```

```
    struct Node * temp = head;
```

```
    for (int i = 1; i < pos - 1 && temp != NULL; i++)
```

```
        temp = temp->next;
```

```
    if (temp == NULL) { printf("position out of range\n");
```

```
        return head; }
```

```
    newNode->next = temp->next;
```

```
    newNode->prev = temp;
```

```
    if (temp->next != NULL) temp->next->prev = newNode;
```

```
    temp->next = newNode;
```

```
    return head;
```

```
}
```



```

struct node * deleteAt position (struct Node * head, int pos) {
    if (head == NULL) return NULL;
    struct Node * temp = head;
    for (int i = 1; i < pos && temp != NULL; i++)
        temp = temp -> next;
    if (temp == NULL) { printf("Position out of range\n");
        return head; }
    if (temp -> prev != NULL) temp -> prev -> next = temp -> next;
    else head = temp -> next;
    if (temp -> next != NULL) temp -> next -> prev = temp -> prev;
    free (temp);
    return head;
}

```

```

void printList (struct Node * head) {
    while (head) { printf("%d <=> ", head -> data); head = head
        printf("NULL\n");
        -> next; }
}

```

```

int main () {

```

```

    struct Node * head = createNode (10);

```

```

    head -> next = createNode (20); head -> next -> prev = head;

```

```

    head -> next -> next = createNode (30); head -> next -> next -> prev =

```

```

    head -> next -> next -> next = createNode (40); head -> next -> next -> next -> prev =

```

```

    head -> next -> next -> next -> next = createNode (50); head -> next -> next -> next -> next -> prev =

```

```

    head -> next -> next -> next -> next -> next = createNode (60);

```

```

    head -> next -> next -> next -> next -> next -> prev = head -> next -> next -> next -> next -> next;

```

```

    printf ("original : ");

```

```

    printList (head);

```

```

head = insertAtPosition(head, 25, 3);
printf("After Insert: "); printList(head);
head = deleteAtPosition(head, 2);
printf("After delete: "); printList(head);
return 0;
}

```

2) stack operations (size = 5).

Answer: Initial stack (index 0 = bottom, index 4 = top since full).

Index :	0	1	2	3	4
value :	22	55	33	66	88

Top (full)

i) Invert the elements in stack.

Index :	0	1	2	3	4
value :	88	66	33	55	22

Top = 22 (index 4)

ii) pop() → removes 22.

Index :	0	1	2	3	4
value :	88	66	33	55	[]

↓
Top = 55

iii) pop() → removes 55

Index :	0	1	2	3	4
value :	88	66	33	[]	[]

↓
Top

iv) pop() → remove 33.

Index :	0	1	2	3	4
value :	88	66	[]	[]	[]

↓
Top

v) push(90)

Index :	0	1	2	3	4
value :	88	66	90	[]	[]

↓
Top

vi) Push (36)

index :	0	1	2	3	4
value :	88	66	90	36	[]

↑
Top

vii) Push (11)

index :	0	1	2	3	4
value :	88	66	90	36	11

↑
Top

viii) Push (88)

Index :	0	1	2	3	4
value :	88	66	90	36	11

↓
Top = 11 (overflow - push rejected)

ix) pop() → removes 11.

Index :	0	1	2	3	4
value :	88	66	90	36	[]

↑
Top = 36

x) Pop() → removes 36.

Index :	0	1	2	3	4
Value :	88	66	90	[]	[]

↑
Top = 90 → final state.